

Предобработка данных и логистическая регрессия для задачи бинарной классификации

Programming assignment

В задании вам будет предложено ознакомиться с основными техниками предобработки данных, а также применить их для обучения модели логистической регрессии. Ответ потребуется загрузить в соответствующую форму в виде 6 текстовых файлов.

Для выполнения задания требуется Python версии 2.7, а также актуальные версии библиотек:

- NumPy: 1.10.4 и выше
- Pandas: 0.17.1 и выше
- Scikit-learn: 0.17 и выше

In []:

```
import pandas as pd
import numpy as np
import matplotlib
from matplotlib import pyplot as plt
matplotlib.style.use('ggplot')
%matplotlib inline
```

Описание датасета

Задача: по 38 признакам, связанных с заявкой на грант (область исследований учёных, информация по их академическому бэкграунду, размер гранта, область, в которой он выдаётся) предсказать, будет ли заявка принята. Датасет включает в себя информацию по 6000 заявкам на гранты, которые были поданы в университете Мельбурна в период с 2004 по 2008 год.

Полную версию данных с большим количеством признаков можно найти на <https://www.kaggle.com/c/unimelb> (<https://www.kaggle.com/c/unimelb>).

In []:

```
data = pd.read_csv('data.csv')
data.shape
```

Выделим из датасета целевую переменную Grant.Status и обозначим её за у. Теперь X обозначает обучающую выборку, y - ответы на ней.

In []:

```
X = data.drop('Grant.Status', 1)
y = data['Grant.Status']
```

Теория по логистической регрессии

После осознания того, какую именно задачу требуется решить на этих данных, следующим шагом при реальном анализе был бы подбор подходящего метода. В данном задании выбор метода было произведён за вас, это логистическая регрессия. Кратко напомним вам используемую модель.

Логистическая регрессия предсказывает вероятности принадлежности объекта к каждому классу. Сумма ответов логистической регрессии на одном объекте для всех классов равна единице.

$$\sum_{k=1}^K \pi_{ik} = 1, \quad \pi_k \equiv P(y_i = k \mid x_i, \theta),$$

где:

- π_{ik} - вероятность принадлежности объекта x_i из выборки X к классу k
- θ - внутренние параметры алгоритма, которые настраиваются в процессе обучения, в случае логистической регрессии - w, b

Из этого свойства модели в случае бинарной классификации требуется вычислить лишь вероятность принадлежности объекта к одному из классов (вторая вычисляется из условия нормировки вероятностей). Эта вероятность вычисляется, используя логистическую функцию:

$$P(y_i = 1 \mid x_i, \theta) = \frac{1}{1 + \exp(-w^T x_i - b)}$$

Параметры w и b находятся, как решения следующей задачи оптимизации (указаны функционалы с L1 и L2 регуляризацией, с которыми вы познакомились в предыдущих заданиях):

L2-regularization:

$$Q(X, y, \theta) = \frac{1}{2} w^T w + C \sum_{i=1}^l \log(1 + \exp(-y_i(w^T x_i + b))) \longrightarrow \min_{w, b}$$

L1-regularization:

$$Q(X, y, \theta) = \sum_{d=1}^D |w_d| + C \sum_{i=1}^l \log(1 + \exp(-y_i(w^T x_i + b))) \longrightarrow \min_{w, b}$$

C - это стандартный гиперпараметр модели, который регулирует то, насколько сильно мы позволяем модели подстраиваться под данные.

Предобработка данных

Из свойств данной модели следует, что:

- все X должны быть числовыми данными (в случае наличия среди них категорий, их требуется некоторым способом преобразовать в вещественные числа)
- среди X не должно быть пропущенных значений (т.е. все пропущенные значения перед применением модели следует каким-то образом заполнить)

Поэтому базовым этапом в предобработке любого датасета для логистической регрессии будет кодирование категориальных признаков, а так же удаление или интерпретация пропущенных значений (при наличии того или другого).

In []:

```
data.head()
```

Видно, что в датасете есть как числовые, так и категориальные признаки. Получим списки их названий:

In []:

```
numeric_cols = ['RFCD.Percentage.1', 'RFCD.Percentage.2', 'RFCD.Percentage.3',  
                'RFCD.Percentage.4', 'RFCD.Percentage.5',  
                'SEO.Percentage.1', 'SEO.Percentage.2', 'SEO.Percentage.3',  
                'SEO.Percentage.4', 'SEO.Percentage.5',  
                'Year.of.Birth.1', 'Number.of.Successful.Grant.1', 'Number.of.Unsuccess  
ful.Grant.1']  
categorical_cols = list(set(X.columns.values.tolist()) - set(numeric_cols))
```

Также в нём присутствуют пропущенные значения. Очевидным решением будет исключение всех данных, у которых пропущено хотя бы одно значение. Сделаем это:

In []:

```
data.dropna().shape
```

Видно, что тогда мы выбросим почти все данные, и такой метод решения в данном случае не работает.

Пропущенные значения можно так же интерпретировать, для этого существует несколько способов, они различаются для категориальных и вещественных признаков.

Для вещественных признаков:

- заменить на 0 (данный признак давать вклад в предсказание для данного объекта не будет)
- заменить на среднее (каждый пропущенный признак будет давать такой же вклад, как и среднее значение признака на датасете)

Для категориальных:

- интерпретировать пропущенное значение, как ещё одну категорию (данный способ является самым естественным, так как в случае категорий у нас есть уникальная возможность не потерять информацию о наличии пропущенных значений; обратите внимание, что в случае вещественных признаков данная информация неизбежно теряется)

Задание 0. Обработка пропущенных значений.

1. Заполните пропущенные вещественные значения в X нулями и средними по столбцам, назовите полученные датафреймы X_real_zeros и X_real_mean соответственно. Для подсчёта средних используйте описанную ниже функцию calculate_means, которой требуется передать на вход вещественные признаки из исходного датафрейма.
2. Все категориальные признаки в X преобразуйте в строки, пропущенные значения требуется также преобразовать в какие-либо строки, которые не являются категориями (например, 'NA'), полученный датафрейм назовите X_cat.

Для объединения выборок здесь и далее в задании рекомендуется использовать функции

```
np.hstack(...)  
np.vstack(...)
```

In []:

```
def calculate_means(numeric_data):  
    means = np.zeros(numeric_data.shape[1])  
    for j in range(numeric_data.shape[1]):  
        to_sum = numeric_data.iloc[:,j]  
        indices = np.nonzero(~numeric_data.iloc[:,j].isnull())[0]  
        correction = np.amax(to_sum[indices])  
        to_sum /= correction  
        for i in indices:  
            means[j] += to_sum[i]  
        means[j] /= indices.size  
        means[j] *= correction  
    return pd.Series(means, numeric_data.columns)
```

In []:

```
# place your code here
```

Преобразование категориальных признаков.

В предыдущей ячейке мы разделили наш датасет ещё на две части: в одной присутствуют только вещественные признаки, в другой только категориальные. Это понадобится нам для отдельной последующей обработке этих данных, а так же для сравнения качества работы тех или иных методов.

Для использования модели регрессии требуется преобразовать категориальные признаки в вещественные. Рассмотрим основной способ преобразования категориальных признаков в вещественные: one-hot encoding. Его идея заключается в том, что мы преобразуем категориальный признак при помощи бинарного кода: каждой категории ставим в соответствие набор из нулей и единиц.

Посмотрим, как данный метод работает на простом наборе данных.

In []:

```
from sklearn.linear_model import LogisticRegression as LR
from sklearn.feature_extraction import DictVectorizer as DV

categorical_data = pd.DataFrame({'sex': ['male', 'female', 'male', 'female'],
                                'nationality': ['American', 'European', 'Asian', 'European']})
print('Исходные данные:\n')
print(categorical_data)
encoder = DV(sparse = False)
encoded_data = encoder.fit_transform(categorical_data.T.to_dict().values())
print('\nЗакодированные данные:\n')
print(encoded_data)
```

Как видно, в первые три колонки оказалась закодированная информация о стране, а во вторые две - о поле. При этом для совпадающих элементов выборки строки будут полностью совпадать. Также из примера видно, что кодирование признаков сильно увеличивает их количество, но полностью сохраняет информацию, в том числе о наличии пропущенных значений (их наличие просто становится одним из бинарных признаков в преобразованных данных).

Теперь применим one-hot encoding к категориальным признакам из исходного датасета. Обратите внимание на общий для всех методов преобработки данных интерфейс. Функция

```
encoder.fit_transform(X)
```

позволяет вычислить необходимые параметры преобразования, впоследствии к новым данным можно уже применять функцию

```
encoder.transform(X)
```

Очень важно применять одинаковое преобразование как к обучающим, так и тестовым данным, потому что в противном случае вы получите непредсказуемые, и, скорее всего, плохие результаты. В частности, если вы отдельно закодируете обучающую и тестовую выборку, то получите вообще говоря разные коды для одних и тех же признаков, и ваше решение работать не будет.

Также параметры многих преобразований (например, рассмотренное ниже масштабирование) нельзя вычислять одновременно на данных из обучения и теста, потому что иначе подсчитанные на тесте метрики качества будут давать смещённые оценки на качество работы алгоритма. Кодирование категориальных признаков не считает на обучающей выборке никаких параметров, поэтому его можно применять сразу к всему датасету.

In []:

```
encoder = DV(sparse = False)
X_cat_oh = encoder.fit_transform(X_cat.T.to_dict().values())
```

Для построения метрики качества по результату обучения требуется разделить исходный датасет на обучающую и тестовую выборки.

Обращаем внимание на заданный параметр для генератора случайных чисел: `random_state`. Так как результаты на обучении и тесте будут зависеть от того, как именно вы разделите объекты, то предлагается использовать заранее определённое значение для получения результатов, согласованных с ответами в системе проверки заданий.

In []:

```
from sklearn.cross_validation import train_test_split

(X_train_real_zeros,
 X_test_real_zeros,
 y_train, y_test) = train_test_split(X_real_zeros, y,
                                     test_size=0.3,
                                     random_state=0)

(X_train_real_mean,
 X_test_real_mean) = train_test_split(X_real_mean,
                                     test_size=0.3,
                                     random_state=0)

(X_train_cat_oh,
 X_test_cat_oh) = train_test_split(X_cat_oh,
                                    test_size=0.3,
                                    random_state=0)
```

Описание классов

Итак, мы получили первые наборы данных, для которых выполнены оба ограничения логистической регрессии на входные данные. Обучим на них регрессию, используя имеющийся в библиотеке `sklearn` функционал по подбору гиперпараметров модели

```
optimizer = GridSearchCV(estimator, param_grid)
```

где:

- `estimator` - обучающий алгоритм, для которого будет производиться подбор параметров
- `param_grid` - словарь параметров, ключами которого являются строки-названия, которые передаются алгоритму `estimator`, а значения - набор параметров для перебора

Данный класс выполняет кросс-валидацию обучающей выборки для каждого набора параметров и находит те, на которых алгоритм работает лучше всего. Этот метод позволяет настраивать гиперпараметры по обучающей выборке, избегая переобучения. Некоторые опциональные параметры вызова данного класса, которые нам понадобятся:

- `scoring` - функционал качества, максимум которого ищется кросс валидацией, по умолчанию используется функция `score()` класса `estimator`
- `n_jobs` - позволяет ускорить кросс-валидацию, выполняя её параллельно, число определяет количество одновременно запущенных задач
- `cv` - количество фолдов, на которые разбивается выборка при кросс-валидации

После инициализации класса `GridSearchCV`, процесс подбора параметров запускается следующим методом:

```
optimizer.fit(X, y)
```

На выходе для получения предсказаний можно пользоваться функцией

```
optimizer.predict(X)
```

для меток или

```
optimizer.predict_proba(X)
```

для вероятностей (в случае использования логистической регрессии).

Также можно напрямую получить оптимальный класс `estimator` и оптимальные параметры, так как они являются атрибутами класса `GridSearchCV`:

- `best_estimator_` - лучший алгоритм
- `best_params_` - лучший набор параметров

Класс логистической регрессии выглядит следующим образом:

```
estimator = LogisticRegression(penalty)
```

где `penalty` принимает либо значение 'l2', либо 'l1'. По умолчанию устанавливается значение 'l2', и везде в задании, если об этом не оговорено особо, предполагается использование логистической регрессии с L2-регуляризацией.

Задание 1. Сравнение способов заполнения вещественных пропущенных значений.

1. Составьте две обучающие выборки из вещественных и категориальных признаков: в одной вещественные признаки, где пропущенные значения заполнены нулями, в другой - средними. Рекомендуется записывать в выборки сначала вещественные, а потом категориальные признаки.
2. Обучите на них логистическую регрессию, подбирая параметры из заданной сетки `param_grid` по методу кросс-валидации с числом фолдов `cv=3`. В качестве оптимизируемой функции используйте заданную по умолчанию.
3. Постройте два графика оценок точности $\pm$ их стандартного отклонения в зависимости от гиперпараметра и убедитесь, что вы действительно нашли её максимум. Также обратите внимание на большую дисперсию получаемых оценок (уменьшить её можно увеличением числа фолдов `cv`).
4. Получите две метрики качества AUC ROC на тестовой выборке и сравните их между собой. Какой способ заполнения пропущенных вещественных значений работает лучше? В дальнейшем для выполнения задания в качестве вещественных признаков используйте ту выборку, которая даёт лучшее качество на тесте.
5. Передайте два значения AUC ROC (сначала для выборки, заполненной средними, потом для выборки, заполненной нулями) в функцию `write_answer_1` и запустите её. Полученный файл является ответом на 1 задание.

Информация для интересующихся: вообще говоря, не вполне логично оптимизировать на кросс-валидации заданный по умолчанию в классе логистической регрессии функционал энтропии, а измерять на тесте AUC ROC, но это, как и ограничение размера выборки, сделано для ускорения работы процесса кросс-валидации.

In []:

```
from sklearn.linear_model import LogisticRegression
from sklearn.grid_search import GridSearchCV
from sklearn.metrics import roc_auc_score

def plot_scores(optimizer):
    scores = [[item[0]['C'],
               item[1],
               (np.sum((item[2]-item[1])**2)/(item[2].size-1))*0.5] for item in optimizer.grid_scores_]
    scores = np.array(scores)
    plt.semilogx(scores[:,0], scores[:,1])
    plt.fill_between(scores[:,0], scores[:,1]-scores[:,2],
                    scores[:,1]+scores[:,2], alpha=0.3)
    plt.show()

def write_answer_1(auc_1, auc_2):
    auc = (auc_1 + auc_2)/2
    with open("preprocessing_lr_answer1.txt", "w") as fout:
        fout.write(str(auc))

param_grid = {'C': [0.01, 0.05, 0.1, 0.5, 1, 5, 10]}
cv = 3

# place your code here
```

Масштабирование вещественных признаков.

Попробуем как-то улучшить качество классификации. Для этого посмотрим на сами данные:

In []:

```
from pandas.tools.plotting import scatter_matrix

data_numeric = pd.DataFrame(X_train_real_zeros, columns=numeric_cols)
list_cols = ['Number.of.Successful.Grant.1', 'SEO.Percentage.2', 'Year.of.Birth.1']
scatter_matrix(data_numeric[list_cols], alpha=0.5, figsize=(10, 10))
plt.show()
```

Как видно из графиков, разные признаки очень сильно отличаются друг от друга по модулю значений (обратите внимание на диапазоны значений осей x и y). В случае обычной регрессии это никак не влияет на качество обучаемой модели, т.к. у меньших по модулю признаков будут большие веса, но при использовании регуляризации, которая штрафует модель за большие веса, регрессия, как правило, начинает работать хуже.

В таких случаях всегда рекомендуется делать стандартизацию (масштабирование) признаков, для того чтобы они меньше отличались друг от друга по модулю, но при этом не нарушались никакие другие свойства признакового пространства. При этом даже если итоговое качество модели на тесте уменьшается, это повышает её интерпретируемость, потому что новые веса имеют смысл "значимости" данного признака для итоговой классификации.

Стандартизация осуществляется посредством вычета из каждого признака среднего значения и нормировки на выборочное стандартное отклонение:

$$x_{id}^{scaled} = \frac{x_{id} - \mu_d}{\sigma_d}, \quad \mu_d = \frac{1}{N} \sum_{i=1}^l x_{id}, \quad \sigma_d = \sqrt{\frac{1}{N-1} \sum_{i=1}^l (x_{id} - \mu_d)^2}$$

Задание 1.5. Масштабирование вещественных признаков.

1. По аналогии с вызовом one-hot encoder примените масштабирование вещественных признаков для обучающих и тестовых выборок `X_train_real_zeros` и `X_test_real_zeros`, используя класс

`StandardScaler`

и методы

```
StandardScaler.fit_transform(...)
StandardScaler.transform(...)
```

2. Сохраните ответ в переменные `X_train_real_scaled` и `X_test_real_scaled` соответственно

In []:

```
from sklearn.preprocessing import StandardScaler  
  
# place your code here
```

Сравнение признаков пространств.

Построим такие же графики для преобразованных данных:

In []:

```
data_numeric_scaled = pd.DataFrame(X_train_real_scaled, columns=numeric_cols)  
list_cols = ['Number.of.Successful.Grant.1', 'SEO.Percentage.2', 'Year.of.Birth.1']  
scatter_matrix(data_numeric_scaled[list_cols], alpha=0.5, figsize=(10, 10))  
plt.show()
```

Как видно из графиков, мы не поменяли свойства признакового пространства: гистограммы распределений значений признаков, как и их scatter-plots, выглядят так же, как и до нормировки, но при этом все значения теперь находятся примерно в одном диапазоне, тем самым повышая интерпретабельность результатов, а также лучше сочетаясь с идеологией регуляризации.

Задание 2. Сравнение качества классификации до и после масштабирования вещественных признаков.

1. Обучите ещё раз регрессию и гиперпараметры на новых признаках, объединив их с закодированными категориальными.
2. Проверьте, был ли найден оптимум ассигасу по гиперпараметрам во время кроссвалидации.
3. Получите значение ROC AUC на тестовой выборке, сравните с лучшим результатом, полученными ранее.
4. Запишите полученный ответ в файл при помощи функции `write_answer_2`.

In []:

```
def write_answer_2(auc):  
    with open("preprocessing_lr_answer2.txt", "w") as fout:  
        fout.write(str(auc))  
  
# place your code here
```

Балансировка классов.

Алгоритмы классификации могут быть очень чувствительны к несбалансированным классам. Рассмотрим пример с выборками, сэмплированными из двух гауссиан. Их мат. ожидания и матрицы ковариации заданы так, что истинная разделяющая поверхность должна проходить параллельно оси x . Поместим в обучающую выборку 20 объектов, сэмплированных из 1-й гауссианы, и 10 объектов из 2-й. После этого обучим на них линейную регрессию, и построим на графиках объекты и области классификации.

In []:

```

np.random.seed(0)
"""Сэмплируем данные из первой гауссианы"""
data_0 = np.random.multivariate_normal([0,0], [[0.5,0],[0,0.5]], size=40)
"""И из второй"""
data_1 = np.random.multivariate_normal([0,1], [[0.5,0],[0,0.5]], size=40)
"""На обучение берём 20 объектов из первого класса и 10 из второго"""
example_data_train = np.vstack([data_0[:20,:], data_1[:10,:]])
example_labels_train = np.concatenate([np.zeros((20)), np.ones((10))])
"""На тест - 20 из первого и 30 из второго"""
example_data_test = np.vstack([data_0[20:,:], data_1[10:,:]])
example_labels_test = np.concatenate([np.zeros((20)), np.ones((30))])
"""Задаём координатную сетку, на которой будем вычислять область классификации"""
xx, yy = np.meshgrid(np.arange(-3, 3, 0.02), np.arange(-3, 3, 0.02))
"""Обучаем регрессию без балансировки по классам"""
optimizer = GridSearchCV(LogisticRegression(), param_grid, cv=cv, n_jobs=-1)
optimizer.fit(example_data_train, example_labels_train)
"""Строим предсказания регрессии для сетки"""
Z = optimizer.predict(np.c_[xx.ravel(), yy.ravel()]).reshape(xx.shape)
plt.pcolormesh(xx, yy, Z, cmap=plt.cm.Pastel2)
plt.scatter(data_0[:,0], data_0[:,1], color='red')
plt.scatter(data_1[:,0], data_1[:,1], color='blue')
"""Считаем AUC"""
auc_wo_class_weights = roc_auc_score(example_labels_test, optimizer.predict_proba(examp
le_data_test)[:,-1])
plt.title('Without class weights')
plt.show()
print('AUC: %f'%auc_wo_class_weights)
"""Для второй регрессии в LogisticRegression передаём параметр class_weight='balance
d'"""
optimizer = GridSearchCV(LogisticRegression(class_weight='balanced'), param_grid, cv=cv
, n_jobs=-1)
optimizer.fit(example_data_train, example_labels_train)
Z = optimizer.predict(np.c_[xx.ravel(), yy.ravel()]).reshape(xx.shape)
plt.pcolormesh(xx, yy, Z, cmap=plt.cm.Pastel2)
plt.scatter(data_0[:,0], data_0[:,1], color='red')
plt.scatter(data_1[:,0], data_1[:,1], color='blue')
auc_w_class_weights = roc_auc_score(example_labels_test, optimizer.predict_proba(examp
le_data_test)[:,-1])
plt.title('With class weights')
plt.show()
print('AUC: %f'%auc_w_class_weights)

```

Как видно, во втором случае классификатор находит разделяющую поверхность, которая ближе к истинной, т.е. меньше переобучается. Поэтому на сбалансированность классов в обучающей выборке всегда следует обращать внимание.

Посмотрим, сбалансированны ли классы в нашей обучающей выборке:

In []:

```

print(np.sum(y_train==0))
print(np.sum(y_train==1))

```

Видно, что нет.

Исправить ситуацию можно разными способами, мы рассмотрим два:

- давать объектам миноритарного класса больший вес при обучении классификатора (рассмотрен в примере выше)
- досэмплировать объекты миноритарного класса, пока число объектов в обоих классах не сравняется

Задание 3. Балансировка классов.

1. Обучите логистическую регрессию и гиперпараметры с балансировкой классов, используя веса (параметр `class_weight='balanced'` регрессии) на отмасштабированных выборках, полученных в предыдущем задании. Убедитесь, что вы нашли максимум ассигасы по гиперпараметрам.
2. Получите метрику ROC AUC на тестовой выборке.
3. Сбалансируйте выборку, досэмплировав в неё объекты из меньшего класса. Для получения индексов объектов, которые требуется добавить в обучающую выборку, используйте следующую комбинацию вызовов функций:

```
np.random.seed(0)
indices_to_add = np.random.randint(...)
X_train_to_add = X_train[y_train.as_matrix() == 1,:][indices_to_add,:]
```

После этого добавьте эти объекты в начало или конец обучающей выборки. Дополните соответствующим образом вектор ответов.

4. Получите метрику ROC AUC на тестовой выборке, сравните с предыдущим результатом.
5. Внесите ответы в выходной файл при помощи функции `write_answer_3`, передав в неё сначала ROC AUC для балансировки весами, а потом балансировки выборки вручную.

In []:

```
def write_answer_3(auc_1, auc_2):
    auc = (auc_1 + auc_2) / 2
    with open("preprocessing_lr_answer3.txt", "w") as fout:
        fout.write(str(auc))
```

place your code here

Стратификация выборок.

Рассмотрим ещё раз пример с выборками из нормальных распределений. Посмотрим ещё раз на качество классификаторов, получаемое на тестовых выборках:

In []:

```
print('AUC ROC for classifier without weighted classes', auc_wo_class_weights)
print('AUC ROC for classifier with weighted classes: ', auc_w_class_weights)
```

Насколько эти цифры реально отражают качество работы алгоритма, если учесть, что тестовая выборка так же несбалансирована, как обучающая? При этом мы уже знаем, что алгоритм логистической регрессии чувствителен к балансировке классов в обучающей выборке, т.е. в данном случае на тесте он будет давать заведомо заниженные результаты. Метрика классификатора на тесте имела бы гораздо больший смысл, если бы объекты были разделены в выборках поровну: по 20 из каждого класса на обучение и на тесте. Переформируем выборки и подсчитаем новые ошибки:

In []:

```

"""Разделим данные по классам поровну между обучающей и тестовой выборками"""
example_data_train = np.vstack([data_0[:20,:], data_1[:20,:]])
example_labels_train = np.concatenate([np.zeros((20)), np.ones((20))])
example_data_test = np.vstack([data_0[20:,:], data_1[20:,:]])
example_labels_test = np.concatenate([np.zeros((20)), np.ones((20))])
"""Обучим классификатор"""
optimizer = GridSearchCV(LogisticRegression(class_weight='balanced'), param_grid, cv=cv,
n_jobs=-1)
optimizer.fit(example_data_train, example_labels_train)
Z = optimizer.predict(np.c_[xx.ravel(), yy.ravel()]).reshape(xx.shape)
plt.pcolormesh(xx, yy, Z, cmap=plt.cm.Pastel2)
plt.scatter(data_0[:,0], data_0[:,1], color='red')
plt.scatter(data_1[:,0], data_1[:,1], color='blue')
auc_stratified = roc_auc_score(example_labels_test, optimizer.predict_proba(example_data_test)[:,1])
plt.title('With class weights')
plt.show()
print('AUC ROC for stratified samples: ', auc_stratified)

```

Как видно, после данной процедуры ответ классификатора изменился незначительно, а вот качество увеличилось. При этом, в зависимости от того, как вы разбили изначально данные на обучение и тест, после сбалансированного разделения выборок итоговая метрика на тесте может как увеличиться, так и уменьшиться, но доверять ей можно значительно больше, т.к. она построена с учётом специфики работы классификатора. Данный подход является частным случаем т.н. метода стратификации.

Задание 4. Стратификация выборки.

1. По аналогии с тем, как это было сделано в начале задания, разбейте выборки `X_real_zeros` и `X_cat_oh` на обучение и тест, передавая в функцию

```
train_test_split(...)
```

дополнительно параметр

```
stratify=y
```

Также обязательно передайте в функцию переменную `random_state=0`.

2. Выполните масштабирование новых вещественных выборок, обучите классификатор и его гиперпараметры при помощи метода кросс-валидации, делая поправку на несбалансированные классы при помощи весов. Убедитесь в том, что нашли оптимум ассигасу по гиперпараметрам.
3. Оцените качество классификатора метрике AUC ROC на тестовой выборке.
4. Полученный ответ передайте функции `write_answer_4`

In []:

```
def write_answer_4(auc):
    with open("preprocessing_lr_answer4.txt", "w") as fout:
        fout.write(str(auc))

# place your code here
```

Теперь вы разобрались с основными этапами предобработки данных для линейных классификаторов. Напомним основные этапы:

- обработка пропущенных значений
- обработка категориальных признаков
- стратификация
- балансировка классов
- масштабирование

Данные действия с данными рекомендуется проводить всякий раз, когда вы планируете использовать линейные методы. Рекомендация по выполнению многих из этих пунктов справедлива и для других методов машинного обучения.

Трансформация признаков.

Теперь рассмотрим способы преобразования признаков. Существует достаточно много различных способов трансформации признаков, которые позволяют при помощи линейных методов получать более сложные разделяющие поверхности. Самым базовым является полиномиальное преобразование признаков. Его идея заключается в том, что помимо самих признаков вы дополнительно включаете набор все полиномы степени p , которые можно из них построить. Для случая $p = 2$ преобразование выглядит следующим образом:

$$\phi(x_i) = [x_{i,1}^2, \dots, x_{i,D}^2, x_{i,1}x_{i,2}, \dots, x_{i,D}x_{i,D-1}, x_{i,1}, \dots, x_{i,D}, 1]$$

Рассмотрим принцип работы данных признаков на данных, сэмплированных их гауссиан:

In []:

```
from sklearn.preprocessing import PolynomialFeatures

"""Инициализируем класс, который выполняет преобразование"""
transform = PolynomialFeatures(2)
"""Обучаем преобразование на обучающей выборке, применяем его к тестовой"""
example_data_train_poly = transform.fit_transform(example_data_train)
example_data_test_poly = transform.transform(example_data_test)
"""Обращаем внимание на параметр fit_intercept=False"""
optimizer = GridSearchCV(LogisticRegression(class_weight='balanced', fit_intercept=False),
                           param_grid, cv=cv, n_jobs=-1)
optimizer.fit(example_data_train_poly, example_labels_train)
Z = optimizer.predict(transform.transform(np.c_[xx.ravel(), yy.ravel()])).reshape(xx.shape)
plt.pcolormesh(xx, yy, Z, cmap=plt.cm.Pastel2)
plt.scatter(data_0[:,0], data_0[:,1], color='red')
plt.scatter(data_1[:,0], data_1[:,1], color='blue')
plt.title('With class weights')
plt.show()
```

Видно, что данный метод преобразования данных уже позволяет строить нелинейные разделяющие поверхности, которые могут более тонко подстраиваться под данные и находить более сложные зависимости. Число признаков в новой модели:

In []:

```
print(example_data_train_poly.shape)
```

Но при этом одновременно данный метод способствует более сильной способности модели к переобучению из-за быстрого роста числа признаков с увеличением степени p . Рассмотрим пример с $p = 11$:

In []:

```
transform = PolynomialFeatures(11)
example_data_train_poly = transform.fit_transform(example_data_train)
example_data_test_poly = transform.transform(example_data_test)
optimizer = GridSearchCV(LogisticRegression(class_weight='balanced', fit_intercept=False),
                           param_grid, cv=cv, n_jobs=-1)
optimizer.fit(example_data_train_poly, example_labels_train)
Z = optimizer.predict(transform.transform(np.c_[xx.ravel(), yy.ravel()])).reshape(xx.shape)
plt.pcolormesh(xx, yy, Z, cmap=plt.cm.Pastel2)
plt.scatter(data_0[:,0], data_0[:,1], color='red')
plt.scatter(data_1[:,0], data_1[:,1], color='blue')
plt.title('Corrected class weights')
plt.show()
```

Количество признаков в данной модели:

In []:

```
print(example_data_train_poly.shape)
```

Задание 5. Трансформация вещественных признаков.

1. Реализуйте по аналогии с примером преобразование вещественных признаков модели при помощи полиномиальных признаков степени 2
2. Постройте логистическую регрессию на новых данных, одновременно подобрав оптимальные гиперпараметры. Обращаем внимание, что в преобразованных признаках уже присутствует столбец, все значения которого равны 1, поэтому обучать дополнительно значение b не нужно, его функцию выполняет один из весов w . В связи с этим во избежание линейной зависимости в датасете, в вызов класса логистической регрессии требуется передавать параметр `fit_intercept=False`. Для обучения используйте стратифицированные выборки с балансировкой классов при помощи весов, преобразованные признаки требуется заново отмасштабировать.
3. Получите AUC ROC на тесте и сравните данный результат с использованием обычных признаков.
4. Передайте полученный ответ в функцию `write_answer_5`.

In []:

```
def write_answer_5(auc):  
    with open("preprocessing_lr_answer5.txt", "w") as fout:  
        fout.write(str(auc))  
  
# place your code here
```

Регрессия Lasso.

К логистической регрессии также можно применить L1-регуляризацию (Lasso), вместо регуляризации L2, которая будет приводить к отбору признаков. Вам предлагается применить L1-регуляризацию к исходным признакам и проинтерпретировать полученные результаты (применение отбора признаков к полиномиальным так же можно успешно применять, но в нём уже будет отсутствовать компонента интерпретации, т.к. смысловое значение оригинальных признаков известно, а полиномиальных - уже может быть достаточно нетривиально). Для вызова логистической регрессии с L1-регуляризацией достаточно передать параметр `penalty='l1'` в инициализацию класса.

Задание 6. Отбор признаков при помощи регрессии Lasso.

1. Обучите регрессию Lasso на стратифицированных отмасштабированных выборках, используя балансировку классов при помощи весов.
2. Получите ROC AUC регрессии, сравните его с предыдущими результатами.
3. Найдите номера вещественных признаков, которые имеют нулевые веса в итоговой модели.
4. Передайте их список функции `write_answer_6`.

In []:

```
def write_answer_6(features):  
    with open("preprocessing_lr_answer6.txt", "w") as fout:  
        fout.write(" ".join([str(num) for num in features]))  
  
# place your code here
```